42 Webserv: Complete Implementation Guide

Project Overview

Build a single-process, event-driven HTTP/1.0 server in C++98 using non-blocking sockets with one poll/select/epoll/kqueue loop. Parse HTTP requests incrementally, serve static files, handle POST uploads, DELETE, run CGI programs, enforce configuration limits.

✓ Core Requirements

- Configuration file with NGINX-like syntax
- Non-blocking I/O with single poll() loop
- GET, POST, DELETE methods
- Static file serving with directory listing
- File uploads (multipart/form-data)
- CGI execution (PHP, Python, etc.)
- Multiple listen ports
- Default error pages
- Never crash or hang indefinitely

Section 1: OS / Networking Primitives

🔧 Concept: TCP Sockets

What It Is

A socket is an OS endpoint for network communication. For servers:

1. `socket()` - Create socket file descriptor
2. `bind()` - Attach to IP address and port
3. `listen()` - Mark as passive (ready to accept)
4. `accept()` - Accept incoming client connections
5. read/write - Communicate with clients
6. `close()` - Clean up

Basic Server Socket Setup

```
// Create socket
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Allow address reuse (prevents "Address already in use")
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// Bind to address and port
struct sockaddr_in address;
memset(&address, 0, sizeof(address));
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;  // Listen on all interfaces
address.sin_port = htons(8080);        // Port 8080

if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen for connections (backlog of 128)
if (listen(server_fd, 128) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}

// Accept a client connection
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
```

```
int client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &client_len);
```

Why It Matters

Every HTTP request arrives as bytes on a client socket. Correct socket handling is the foundation of the entire server.

Key Details

- `AF_INET` = IPv4 address family
- `SOCK_STREAM` = TCP (reliable, ordered byte stream)
- `SOCK_DGRAM` = UDP (unreliable datagrams) - not used in this project
- `SO_REUSEADDR` allows rebinding immediately after server restart
- `listen()` backlog = max pending connections queue
- `accept()` blocks until client connects (unless non-blocking)

🔧 Concept: Non-Blocking I/O

What It Is

`O_NONBLOCK` flag makes syscalls (read/write) return immediately instead of blocking waiting for data or buffer space.

Non-blocking mode:

- `read()` returns -1 with errno=EAGAIN if no data available
- `write()` may write fewer bytes than requested if buffer full
- `accept()` returns -1 with errno=EAGAIN if no pending connections

⚠ Critical Project Rule

You CANNOT check errno after read/write to adjust behavior! You MUST use poll() to determine readiness before I/O operations!

Setting Non-Blocking Mode

```
// Get current flags
int flags = fcntl(fd, F_GETFL, 0);
if (flags < 0) {
    perror("fcntl F_GETFL");
    return -1;
}

// Add O_NONBLOCK flag
if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {
    perror("fcntl F_SETFL O_NONBLOCK");
    return -1;
}

// Also set FD_CLOEXEC (close-on-exec) for security
fcntl(fd, F_SETFD, FD_CLOEXEC);
```

MacOS Note

MacOS handles `write()` differently. You MUST use `fcntl()` with these flags only:

- `F_SETFL`
- `O_NONBLOCK`
- `FD_CLOEXEC`

Any other flags are **FORBIDDEN**.

Why Non-Blocking

Prevents slow clients or disk operations from stalling the entire server. The subject REQUIRES non-blocking for correctness and grading.

✕ Wrong Approach (FORBIDDEN)

```
int bytes = read(fd, buffer, size);
if (bytes < 0 && errno == EAGAIN) {  // ✕ FORBIDDEN!
    // Don't check errno!
}
```

✓ Correct Approach

```
// Use poll() to check if fd is readable first
struct pollfd pfd;
pfd.fd = fd;
pfd.events = POLLIN;
```

```
int ret = poll(&pfd, 1, 0);  // Non-blocking poll
if (ret > 0 && (pfd.revents & POLLIN)) {
    int bytes = read(fd, buffer, size);  // Now safe to read
}
```

🔧 Concept: Multiplexing (poll/select/epoll/kqueue)

What It Is

System calls that monitor multiple file descriptors for readiness events:

- **Readable (POLLIN)** - data available to read
- **Writable (POLLOUT)** - buffer space available to write
- **Errors (POLLERR, POLLHUP)** - connection errors or hangups

Project Requirement

✓ Use exactly ONE poll() (or equivalent) for ALL I/O ✓ Monitor server socket(s) AND all client sockets in same loop ✓ Never read/write without poll() indicating readiness

poll() Basic Structure

```
struct pollfd fds[MAX_CLIENTS];
int nfds = 0;

// Add server socket
fds[nfds].fd = server_fd;
fds[nfds].events = POLLIN;  // Monitor for incoming connections
nfds++;

// Add client sockets
for (each client) {
    fds[nfds].fd = client_fd;
    fds[nfds].events = POLLIN;      // Monitor for incoming data
    if (client.has_data_to_send()) {
        fds[nfds].events |= POLLOUT; // Also monitor for write readiness
    }
    nfds++;
}

// Wait for events (timeout in milliseconds, -1 = infinite)
int ret = poll(fds, nfds, 1000);  // 1 second timeout
```

```
    if (ret < 0) {
        perror("poll");
    } else if (ret == 0) {
        // Timeout - check for idle clients
    } else {
        // Check which fds have events
        for (int i = 0; i < nfds; i++) {
            if (fds[i].revents & POLLIN) {
                // fd is readable
            }
            if (fds[i].revents & POLLOUT) {
                // fd is writable
            }
            if (fds[i].revents & (POLLERR | POLLHUP)) {
                // Error or disconnect
            }
        }
    }
```

Poll vs Select vs Epoll vs Kqueue

poll():

- ✓ No fd limit (select limited to 1024)
- ✓ Cleaner API
- ✓ Portable (Linux, MacOS, BSD)
- ✗ O(n) performance

select():

- ✓ Most portable
- ✗ FD_SETSIZE limit (usually 1024)
- ✗ Must rebuild fd_set each iteration
- ✗ O(n) performance

epoll() (Linux only):

- ✓ O(1) for active connections
- ✓ Scalable to 10,000+ connections
- ✗ Linux-specific
- ✓ Edge-triggered mode available

kqueue() (BSD/MacOS):

- ✓ O(1) for active connections
- ✓ More flexible event system
- ✗ BSD/MacOS only

Project Choice

Any of these is acceptable. **poll()** is recommended for simplicity and portability.

Why Multiplexing Matters

Single event loop = single-threaded, deterministic, no race conditions, easier to reason about resource usage. This is what the graders expect.

Section 2: HTTP/1.0 Protocol

HTTP/1.0 Basics

Text-based protocol over TCP. Each message has:

1. Start line (request-line or status-line)
2. Headers (key: value pairs)
3. Empty line (CRLF)
4. Optional body

HTTP Request Format

```
METHOD SP request-target SP HTTP/1.0 CRLF
Host: localhost:8080 CRLF
User-Agent: Mozilla/5.0 CRLF
Content-Length: 13 CRLF
CRLF
Hello, World!
```

Example Request

```
GET /index.html HTTP/1.0\r\n
Host: localhost:8080\r\n
User-Agent: curl/7.68.0\r\n
Accept: */*\r\n
\r\n
```

HTTP Response Format

```
HTTP/1.0 SP status-code SP reason-phrase CRLF
Content-Type: text/html CRLF
Content-Length: 13 CRLF
CRLF
<h1>Hello</h1>
```

Example Response

```
HTTP/1.0 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 20\r\n
Connection: close\r\n
\r\n
<h1>Hello World</h1>
```

HTTP/1.0 vs HTTP/1.1 Differences

HTTP/1.0:

- Connection: close by default (one request per TCP connection)
- No Host header required (but send it anyway)
- No chunked transfer encoding (body size via Content-Length or EOF)
- Simpler but less efficient

HTTP/1.1:

- Connection: keep-alive by default (persistent connections)
- Host header REQUIRED

- Chunked transfer encoding supported
- More complex but efficient

Project Note

Subject says HTTP/1.0 but allows chunked encoding for CGI compatibility. You must decode chunked requests before passing to CGI!

Required HTTP Methods

GET - Retrieve a resource

Request has no body (use query string for parameters)

POST - Submit data to server

Request has body (forms, file uploads, JSON)

DELETE - Remove a resource

Request usually has no body

MUST IMPLEMENT THESE THREE!

Example GET Request

```
GET /api/users?id=42 HTTP/1.0\r\n
Host: localhost:8080\r\n
\r\n
```

Example POST Request

```
POST /upload HTTP/1.0\r\n
Host: localhost:8080\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Content-Length: 27\r\n
\r\n
name=John&email=john@ex.com
```

Example DELETE Request

```
DELETE /files/document.pdf HTTP/1.0\r\n
Host: localhost:8080\r\n
\r\n
```

Important Response Headers

Content-Length: Size of response body in bytes. CRITICAL for client to know when body ends!

Content-Type: MIME type of body (text/html, application/json, image/png, etc.)

Connection:

- "close" = close after response
- "keep-alive" = keep connection open (HTTP/1.1 style)

Date: When response was generated (optional but nice)

Server: Server software name/version (e.g., "Webserv/1.0")

Example Response with Headers

```
HTTP/1.0 200 OK\r\n
Date: Wed, 08 Oct 2025 12:00:00 GMT\r\n
Server: Webserv/1.0\r\n
Content-Type: text/html; charset=UTF-8\r\n
Content-Length: 145\r\n
Connection: close\r\n
\r\n
<!DOCTYPE html>
<html>
<head><title>Test</title></head>
<body><h1>Hello from Webserv!</h1></body>
</html>
```

HTTP Status Codes (YOU MUST IMPLEMENT)

2xx Success

- **200 OK** - Request succeeded
- **201 Created** - Resource created (POST success)
- **204 No Content** - Success but no body to return

3xx Redirection

- **301 Moved Permanently** - Resource moved, update bookmarks
- **302 Found** - Temporary redirect
- **304 Not Modified** - Use cached version

4xx Client Error

- **400 Bad Request** - Malformed request syntax
- **403 Forbidden** - Server refuses (permissions)
- **404 Not Found** - Resource doesn't exist
- **405 Method Not Allowed** - Method not supported for this route
- **413 Payload Too Large** - Body exceeds limit
- **414 URI Too Long** - URL too long

5xx Server Error

- **500 Internal Server Error** - Server crashed/exception
- **501 Not Implemented** - Feature not supported
- **503 Service Unavailable** - Server overloaded

Body Delimiting in HTTP/1.0

Method 1: Content-Length header Server sends exact byte count, client reads that many bytes.

Method 2: Connection close Server closes connection when done sending (EOF marks end). This is HTTP/1.0 default!

Critical for CGI

If CGI doesn't return Content-Length, you must read until EOF (pipe close)!

Section 3: Incremental Parsing & State Machine

Why Incremental Parsing

TCP is a byte stream - NOT a message stream!

- `read()` may return 1 byte
- `read()` may return partial headers
- `read()` may return headers + part of body
- `read()` may return multiple pipelined requests

YOU CANNOT assume whole HTTP messages arrive in one syscall!

Solution: Per-connection state machine + buffering

Connection State Machine

```
enum ConnectionState {
    READING_HEADERS,        // Accumulate until \r\n\r\n found
    HEADERS_PARSED,         // Determine if body exists
    READING_BODY,           // Read Content-Length bytes
    READING_CHUNKED_BODY,   // Decode chunked encoding
    PROCESSING,             // Route request, generate response
    SENDING_RESPONSE,       // Non-blocking write response
    DONE                    // Close or keep-alive
};
```

STATE: READING_HEADERS

Goal: Accumulate bytes until finding "\r\n\r\n" (header terminator)

```
void handle_reading_headers(Connection& conn) {
    char buffer[4096];
    int bytes = read(conn.fd, buffer, sizeof(buffer));

    if (bytes <= 0) {
        if (bytes == 0) {
            // Client closed connection
            conn.state = DONE;
        }
        return;
    }

    // Append to connection's read buffer
    conn.read_buffer.append(buffer, bytes);

    // Search for header terminator
    size_t pos = conn.read_buffer.find("\r\n\r\n");

    if (pos != std::string::npos) {
        // Headers complete!
        std::string headers = conn.read_buffer.substr(0, pos);

        // Remove headers from buffer (keep any body bytes)
        conn.read_buffer.erase(0, pos + 4);

        // Parse request line and headers
        parse_request_line_and_headers(conn, headers);

        conn.state = HEADERS_PARSED;
    }

    // Check for header size limit
    if (conn.read_buffer.size() > MAX_HEADER_SIZE) {
        send_error(conn, 431);  // Request Header Fields Too Large
        conn.state = DONE;
    }
}
```

STATE: HEADERS_PARSED

Goal: Determine if request has a body

```
void handle_headers_parsed(Connection& conn) {
    // Check for Content-Length header
    if (conn.request.hasHeader("Content-Length")) {
        conn.expected_body_size = atoi(conn.request.getHeader("Content-Length").c_s

        // Check against max body size from config
        if (conn.expected_body_size > config.client_max_body_size) {
            send_error(conn, 413);  // Payload Too Large
            conn.state = DONE;
            return;
```

```
        }

        conn.state = READING_BODY;
    }
    // Check for Transfer-Encoding: chunked
    else if (conn.request.getHeader("Transfer-Encoding") == "chunked") {
        conn.state = READING_CHUNKED_BODY;
    }
    // No body (GET, DELETE, etc.)
    else {
        conn.state = PROCESSING;
    }
}
```

STATE: READING_BODY

Goal: Read exactly Content-Length bytes

```
void handle_reading_body(Connection& conn) {
    size_t needed = conn.expected_body_size - conn.body_buffer.size();

    if (needed == 0) {
        // Body complete!
        conn.request.body = conn.body_buffer;
        conn.state = PROCESSING;
        return;
    }

    char buffer[4096];
    size_t to_read = std::min(sizeof(buffer), needed);

    int bytes = read(conn.fd, buffer, to_read);

    if (bytes <= 0) {
        if (bytes == 0) {
            // Client closed before sending full body
            send_error(conn, 400);  // Bad Request
            conn.state = DONE;
        }
        return;
    }

    // Append to body buffer
    conn.body_buffer.append(buffer, bytes);

    // Check if body is complete
    if (conn.body_buffer.size() >= conn.expected_body_size) {
        conn.request.body = conn.body_buffer;
        conn.state = PROCESSING;
    }
}
```

STATE: PROCESSING

Goal: Route request and generate response

```
void handle_processing(Connection& conn) {
    // Route based on method, path, and config
    Route* route = find_route(conn.request.path, config);

    if (!route) {
        send_error(conn, 404);  // Not Found
        conn.state = SENDING_RESPONSE;
        return;
    }

    // Check if method is allowed for this route
    if (!route->is_method_allowed(conn.request.method)) {
        send_error(conn, 405);  // Method Not Allowed
        conn.state = SENDING_RESPONSE;
        return;
    }

    // Handle based on route type
    if (route->is_cgi()) {
        start_cgi(conn, route);
        // State stays PROCESSING until CGI completes
    } else if (conn.request.method == "GET") {
        handle_get(conn, route);
        conn.state = SENDING_RESPONSE;
    } else if (conn.request.method == "POST") {
        handle_post(conn, route);
        conn.state = SENDING_RESPONSE;
    } else if (conn.request.method == "DELETE") {
        handle_delete(conn, route);
        conn.state = SENDING_RESPONSE;
    }
}
```

STATE: SENDING_RESPONSE

Goal: Non-blocking write of response

```
void handle_sending_response(Connection& conn) {
    if (conn.write_queue.empty()) {
        conn.state = DONE;
        return;
    }

    // Get next buffer to send
    std::string& buffer = conn.write_queue.front();

    int bytes = write(conn.fd, buffer.c_str() + conn.write_offset,
```

```
                        buffer.size() - conn.write_offset);

    if (bytes <= 0) {
        if (bytes < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            // Would block - wait for POLLOUT
            return;
        }
        // Error or disconnect
        conn.state = DONE;
        return;
    }

    conn.write_offset += bytes;

    // Check if current buffer is fully sent
    if (conn.write_offset >= buffer.size()) {
        conn.write_queue.pop_front();
        conn.write_offset = 0;
    }
}
```

STATE: DONE

Goal: Clean up or keep-alive

```
void handle_done(Connection& conn) {
    // Check for Connection: keep-alive
    if (conn.request.getHeader("Connection") == "keep-alive" &&
        !conn.read_buffer.empty()) {
        // Pipelined request - reset and start over
        conn.state = READING_HEADERS;
        conn.request.clear();
        conn.response.clear();
        conn.body_buffer.clear();
        return;
    }

    // Close connection
    close(conn.fd);
    remove_connection(conn);
}
```

Section 4: Event Loop - The Heart of the Server

Single Event Loop Responsibilities

1. Accept new connections (listener sockets)
2. Poll all client sockets for readability/writability/errors
3. Poll CGI pipes (non-blocking stdin/stdout)
4. Drive per-connection state machines
5. Enforce timeouts and resource limits
6. Reap finished CGI child processes

Complete Event Loop Pseudocode

```
void run_server() {
    // Setup
    std::vector<int> listener_fds = setup_listeners(config);
    std::vector<struct pollfd> poll_fds;
    std::map<int, Connection> connections;

    // Add listeners to poll
    for (int fd : listener_fds) {
        struct pollfd pfd;
        pfd.fd = fd;
        pfd.events = POLLIN;
        poll_fds.push_back(pfd);
    }

    while (true) {
        // Calculate timeout for idle connection checks
        int timeout_ms = 1000;  // 1 second

        // Poll all fds
        int ret = poll(&poll_fds[0], poll_fds.size(), timeout_ms);

        if (ret < 0) {
            if (errno == EINTR) continue;  // Interrupted by signal
            perror("poll");
            break;
        }

        if (ret == 0) {
            // Timeout - check for idle connections
            check_timeouts(connections);
            continue;
        }

        // Process events
        for (size_t i = 0; i < poll_fds.size(); i++) {
            if (poll_fds[i].revents == 0) continue;
```

```
            int fd = poll_fds[i].fd;

            // Is this a listener socket?
            if (is_listener(fd, listener_fds)) {
                // Accept new connections
                accept_clients(fd, connections, poll_fds);
                continue;
            }

            // Must be a client connection
            Connection& conn = connections[fd];

            // Handle errors/hangups
            if (poll_fds[i].revents & (POLLERR | POLLHUP | POLLNVAL)) {
                close_connection(conn, connections, poll_fds, i);
                i--;  // Adjust index after removal
                continue;
            }

            // Handle readable
            if (poll_fds[i].revents & POLLIN) {
                handle_read(conn);
                update_last_activity(conn);
            }

            // Handle writable
            if (poll_fds[i].revents & POLLOUT) {
                handle_write(conn);
                update_last_activity(conn);
            }

            // Update poll events based on connection state
            update_poll_events(conn, poll_fds[i]);

            // Check if connection should be closed
            if (conn.state == DONE) {
                close_connection(conn, connections, poll_fds, i);
                i--;
            }
        }

        // Reap finished CGI processes
        reap_cgi_children();

        // Periodic maintenance
        check_resource_limits(connections, poll_fds);
    }
}
```

📝 42 Specific Notes - CRITICAL RULES

⚠ One poll() for ALL I/O (server + clients + CGI) ⚠ Never read/write without poll() readiness ⚠ Never check errno after read/write ⚠ fork() ONLY for CGI ⚠ Non-blocking mode on ALL fds ⚠ Functions max 25 lines (norm compliance) ⚠ C++98 only (no C++11 features) ⚠ Must handle client disconnects gracefully ⚠ Must survive stress tests ⚠ Must serve multiple ports ⚠ Configuration file required

 Allowed System Calls

socket, bind, listen, accept, connect setsockopt, getsockname, getpeername htons, htonl, ntohs, ntohl getaddrinfo, freeaddrinfo, getprotobyname select, poll, epoll (epoll_create, epoll_ctl, epoll_wait) kqueue (kqueue, kevent) read, write, send, recv open, close, stat, access opendir, readdir, closedir fcntl (MacOS: only F_SETFL, O_NONBLOCK, FD_CLOEXEC) fork, execve, waitpid, kill, signal pipe, dup